Guidelines and Submission Rules

Everything you need to know to take part, build, submit and be judged

Qiskit Fall Fest Hackathon · Oct 9, 2026 · IISER Bhopal (EECS Club and Physics Club)

> **The one-line version.** Build something that works, explain every line of it, compare it honestly against a classical alternative, and put it all in a GitHub repository that runs from a clean start.

1. Event at a glance

Item	Details
Event	Qiskit Fall Fest Hackathon, hosted by IISER Bhopal (EECS Club and Physics Club)
Kick-off	Oct 9, 2026 (Vatsal)
Duration	One week to build
Submission deadline	[SUBMISSION DEADLINE - to be confirmed by organisers]
Team size	[TEAM SIZE - to be confirmed by organisers]
Where to submit	[SUBMISSION FORM / CHANNEL LINK]
Questions	[ORGANISER CONTACT EMAIL]
Tools	Laptop and AI assistants. Everything can be done on a simulator (Qiskit Aer or statevector tools).

2. Choosing a project

You have one week, a laptop, and AI assistants. The ten problem statements are suggestions, not a menu you must order from. You may pick one, extend one, combine several, or invent something more ambitious. Original, advanced and cross-track projects are welcome and will be judged on the same criteria.

- **Six tracks, ten suggested problems:** Quantum chemistry (Problems 1-2), Quantum machine learning (3-4), QAOA and optimisation (5-6), Quantum error correction (7-8), Quantum key distribution (9), Post-quantum cryptography (10).
- **Core tasks are the minimum, not the ceiling.** Stretch goals, combined tracks and original ideas score on ambition and originality.
- **Own idea?** If you have your own idea, such as a quantum game, an algorithm tutor, a benchmark of a quantum algorithm from our workshop (Grover, phase estimation, the QFT), or a tool that makes any of the above easier for the next student, propose it. Write three sentences on the problem, the method, and how you will know it worked, and check with the organisers early in the week.
- **Combining tracks?** Get the smallest version of each part working separately first, then connect them. A small, honest, working combination beats an ambitious one that does not run.

3. What you must submit

Every team submits the following. Missing items cost points, because each one maps to a judging criterion.

#	Deliverable	Requirement
1	**GitHub repository**	All code in one public repository (or private with the organisers added as collaborators). Runs from a clean start, with the versions of all packages listed.
2	**Write-up PDF**	One PDF of two to four pages, stored in the repo root as `WRITEUP.pdf`. Sections: Problem, Method, Results, and an honest Limits section.
3	**Main-result figure**	At least one figure that shows the main result, included in the write-up and saved in `figures/`.
4	**AI-use note**	Short note: which tools you used, one thing the AI got wrong, and how you caught it. Included in the write-up and/or as `AI_USE_NOTE.md`.
5	**Demo**	Five-minute demo followed by questions. Every team member must be able to take questions.

Bonus. Bonus points are available for a hardware run on a real IBM backend, with a clear comparison against the simulator. Include the backend name, the date of the run, the job ID, and a side-by-side comparison with your simulator result.

4. Repository rules

Your repository is the main artefact judges will run and read. Use the structure in `submission-template/` as a starting point.

Required contents

Path	Purpose
`README.md`	Team name and members, chosen problem or project title, one-paragraph summary, exact commands to reproduce your main result, and expected runtime.
`requirements.txt`	Every package with its version pinned (`package==x.y.z`), plus the Python version.
`WRITEUP.pdf`	Your two-to-four page write-up.
`AI_USE_NOTE.md`	The AI-use note (may also appear inside the write-up).
`src/` or notebooks	Your code. Readable, commented where it is not obvious.
`figures/`	All figures used in the write-up, including the main-result figure.
`tests/`	Tests or checks against an exact or classical reference (recommended; supports the correctness criterion).

Reproducibility: a clean start

1. Create a fresh virtual environment (for example `python -m venv .venv`).
2. Run `pip install -r requirements.txt` and nothing else.
3. Run the commands in your README. The main result and main figure must be produced without manual edits.
4. Fix random seeds so results can be reproduced, and state in the README which results are statistical and how many runs they average over.
5. If any step needs an IBM Quantum account or API token, say so, and **never commit your token**. Use an environment variable and provide a simulator-only path that works without it.

6 Before submitting, test steps 1-3 on a machine or folder that is not your development setup.

Listing package versions

Run `pip freeze > requirements.txt` inside your project environment and trim it to the packages you actually use, or write them by hand. Qiskit and its add-on packages change quickly between releases, so exact versions matter. Also record your Python version in the README.

5. The write-up (2 to 4 pages)

Section	What to include
Problem	The problem you chose or invented, why it matters, and what a successful result looks like.
Method	What you built and how: circuits, encodings, parameters, optimisers, noise models. Enough detail that a reader could rebuild it.
Results	Your main result with the key figure, numbers with sensible error bars, and the comparison against an exact or classical reference.
Honest limits	What does not work, what you did not test, assumptions, which parts are a simulation shortcut, and where a classical method does as well or better. No unsupported claims of quantum advantage.
AI-use note	Tools used, one thing the AI got wrong, how you caught it.

Keep to four pages. Figures and tables count toward the page limit. Put extra material in the repository and link to it.

6. Using AI

> AI assistants are welcome at every step. The AI-use note and the live questions exist because the person submitting is responsible for the result: you should be able to explain each circuit, formula and plot, and show how you verified it.

- AI use is welcome at every step: ideas, code, debugging, writing.
- The AI-use note must be specific. **"The AI sometimes makes mistakes" is not acceptable.** Name one real error (a wrong gate, a wrong formula, a hallucinated function, a misleading explanation) and describe the check that caught it, such as a test, a comparison with exact diagonalisation, or a known limiting case.
- In the live questions, any team member may be asked to explain any circuit, formula or plot. If you cannot explain or verify code, it will cost you marks under *Understanding and insight* and *Correctness and verification*.
- Do not present AI-generated text or results as verified when you have not verified them.

7. Honesty rules

- **Compare against a classical baseline.** Every result needs a fair classical comparison with similar resources (for example a similar number of parameters).
- **Do not claim a quantum speedup or advantage** unless your measurements actually show one under a fair comparison. Results that say "no advantage, and here is why" are valued.

- **Say what is simulated.** Distinguish a physical circuit simulation from a statistical shortcut, and a noiseless simulator from a noise model from real hardware.
- **Report negative and weak results.** A clear failure analysis scores better than hidden weak results.
- **Cryptography code is for learning only.** Toy schemes (Track 6, and QKD post-processing) must never be used to protect real data, and you must say so in your README.

8. The demo and live questions

- **Five minutes** for the demo, then questions. Keep to time.
- Tell a clear story: the problem, what you built, the main result, and what it does not show.
- Show something running, or a recorded run of the main result as a fallback.
- **Every team member must be present and able to take questions.**
- Have the repository open and runnable on the day. Judges may ask you to run or step through a part of the code.

9. How you are judged

The scoring below applies to every problem and to original projects alike.

Criterion	Points	What judges look for
Correctness and verification	30	Results checked against an exact or classical reference, tests, sensible error bars
Understanding and insight	25	Clear explanation of why the method works or fails, shown in the write-up and in the live questions
Honest comparison and limits	20	A fair classical baseline, stated assumptions, no unsupported claims of quantum advantage
Ambition and originality	15	Stretch goals, combined tracks, or a new idea beyond the suggested statements
Clarity of demo and code	10	Readable, reproducible code and a clear five-minute story
Total	**100**	Bonus points are available on top for a real IBM hardware run (see Bonus).

Bonus: Bonus points are available for a hardware run on a real IBM backend, with a clear comparison against the simulator.

10. Integrity and conduct

- Work must be produced by your team during the hackathon week. Cite any code, data, papers or tutorials you build on (for example the IBM SQD tutorials), and make clear what you added.
- Using open-source libraries (Qiskit, PySCF, ldpc, stim and so on) is expected. Importing a library call that does the whole task is allowed, but where a problem says to build something yourself, building it yourself is what earns the marks.
- Do not copy another team's work. Do not share API tokens, credentials or personal data in your repository.

- Be respectful to participants, volunteers and judges. Organisers may disqualify submissions that break these rules.

11. Final checklist

Tick every box before you submit.

Done	Item
[ ]	Repository runs from a clean start; `requirements.txt` has pinned versions and README states the Python version
[ ]	README gives exact commands and expected runtime for the main result
[ ]	`WRITEUP.pdf` is 2-4 pages with Problem, Method, Results and Honest limits
[ ]	At least one figure shows the main result
[ ]	Results are checked against an exact or classical reference, and a classical baseline is included
[ ]	AI-use note names the tools, one specific AI error, and how you caught it
[ ]	No API tokens or secrets in the repository or git history
[ ]	Hardware run (optional bonus) is compared clearly against the simulator
[ ]	Every team member can explain every circuit, formula and plot, and can take questions in the demo
[ ]	Submitted via [SUBMISSION FORM / CHANNEL LINK] before [SUBMISSION DEADLINE - to be confirmed by organisers]

12. Getting help

Check with the organisers early in the week if you are unsure about scope, a custom idea, or any rule above: [ORGANISER CONTACT EMAIL].